

Chapter 7

Implementing a lock

In this chapter we show how lock taking and lock release, as done by Java's `synchronized` statement or `ReentrantLock` class, can be implemented using low-level compare-and-set operations.

The purpose of this is two-fold. First, it shows in detail what actually goes on when taking and releasing a standard Java lock. Secondly, it shows how the high-level mechanism is implemented in terms of the lower-level compare-and-set mechanism which is directly implemented in hardware instructions such as `lock/cmpxchg` in the Intel/AMD x86 instruction set or `casal` in Arm64.

7.1 What does a lock do

What we shall actually implement is a `ReentrantLock` from the `java.util.concurrent.locks` package, because the Java `synchronized` statement essentially uses a reentrant lock. The statement

```
synchronized (...) {  
    ... critical section ...  
}
```

will first attempt to take the lock, then execute the critical section, and finally release the lock, like this:

```
lock.lock();  
... critical section ...  
lock.unlock();
```

To be even more precise, it will always release the lock regardless of whether the critical section terminates normally, throws an exception, or returns via a `return` statement. The actual encoding of `synchronized(...)` uses a Java `finally` block to achieve this:

```
lock.lock();  
try {  
    ... critical section ...  
} finally {  
    lock.unlock();  
}
```

We shall implement our version of `ReentrantLock` in four steps, because the lock taking method `lock.lock()` has several distinct aspects, just as in Java's `synchronized` statement:

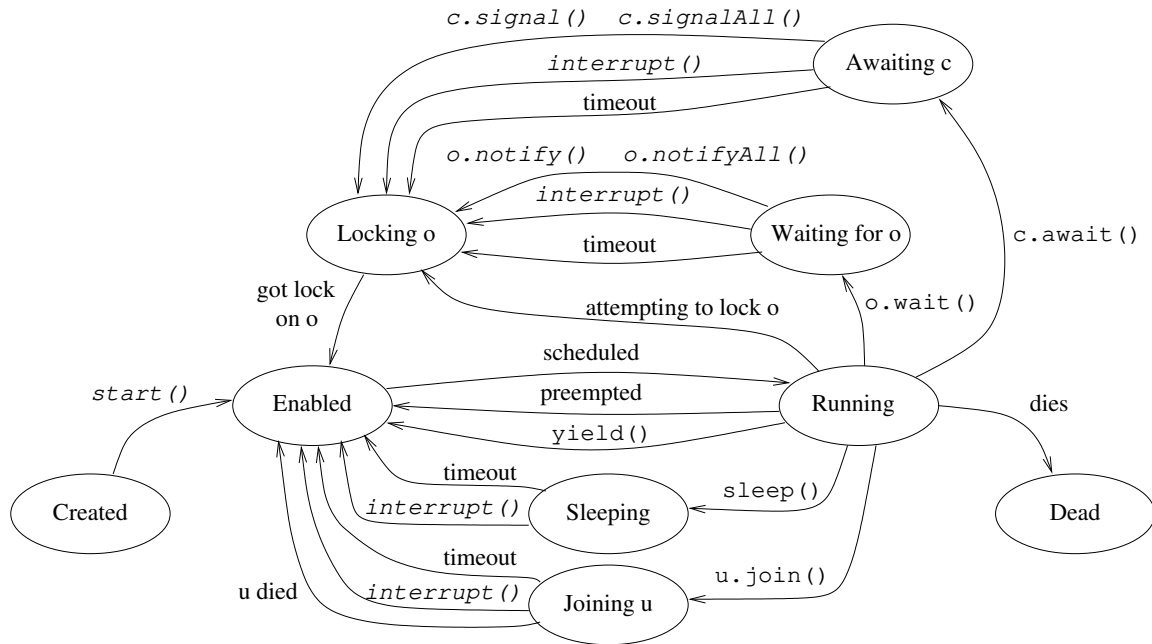


Figure 7.1: The states of a Java thread. A thread’s transition to another state may be caused by the thread itself (*monospace* font), possibly by another thread (*slanted monospace* font), and by timeouts and other actions. The *o* denotes an object whose intrinsic lock is being used; the *c* is a Condition. Augmented from [13].

- Atomically check that the lock is available, and if so take it and let the thread execution continue.
- If the lock is not available, park (block) the calling thread, moving it from the Enabled state to the Locking state shown in figure 7.1. The calling thread will remain in the Locking state until some other thread releases the lock and it manages to get it.
- If the lock is already held by the calling thread, increment the number of times the lock has been taken and let the thread execution continue.

The last point makes the lock reentrant, so that a thread can take the same lock multiple times. If it could not, then a recursive synchronized method would deadlock. In Java, it is the norm for locks and synchronized to be reentrant.

The `lock.unlock()` call should do the following:

- Atomically check that this thread is holding the lock, and if so, decrement the number of times the lock has been taken, and if this number became zero, set the lock status to available. If this thread was not holding the lock, throw an exception.

To keep the three aspects of `lock.lock()` apart, and to introduce the complications one at time, we shall make four different lock implementations, ending with one that mimics Java’s ReentrantLock:

- A** `SimpleTryLock`, a basic lock with non-blocking `tryLock` method, and non-reentrant so the lock can be taken only once by a thread.
- B** `ReentrantTryLock`, a basic lock with non-blocking `tryLock` method, but reentrant so the lock can be taken multiple times by a thread.
- C** `SimpleLock`, with blocking `lock` method that parks (blocks) the thread in case another thread holds the lock, and non-reentrant so the lock can be taken only once by a thread.
- D** `MyReentrantLock`, with blocking `lock` method that parks (blocks) the thread in case another thread holds the lock, and reentrant so the lock can be taken multiple times by the same thread. This mimics `ReentrantLock` from package `java.util.concurrent.locks`.

We have two dimensions of functionality, where one is non-blocking (A and B) versus blocking (C and D), and the other is non-reentrant (A and C) versus reentrant (B and D). Only the final implementation (D) has both the desired functionalities, and implementations A, B and C are just steps toward that.

	Non-reentrant	Reentrant
Non-blocking	(A) <code>SimpleTryLock</code> , section 7.2	(B) <code>ReentrantTryLock</code> , section 7.3
Blocking	(C) <code>SimpleLock</code> , section 7.4	(D) <code>MyReentrantLock</code> , section 7.5

7.2 (A) `SimpleTryLock`: non-blocking and non-reentrant

The `SimpleTryLock` has a single field called `holder` which is an `AtomicReference` to a `Thread` object. If the lock is available (not held by any thread), then `holder` is null. If some thread holds the lock, then `holder` refers to the `Thread` object corresponding to that thread:

```
class SimpleTryLock {
    private final AtomicReference<Thread> holder
        = new AtomicReference<Thread>();

    public boolean tryLock() {
        final Thread current = Thread.currentThread();
        return holder.compareAndSet(null, current);
    }

    public void unlock() {
        final Thread current = Thread.currentThread();
        if (!holder.compareAndSet(current, null))
            throw new RuntimeException("Not lock holder");
    }
}
```

Method `tryLock()` assigns the calling thread's `Thread` object to `current`, then uses `compareAndSet` on the `holder` field to try to take the lock: Atomically test whether the lock is available (when `holder` is null) and if so, set `holder` to `current` and return true; otherwise change nothing and return false, having failed to take the lock.

Method `unlock()` assigns the calling thread's `Thread` object to `current`, then uses `compareAndSet` on the `holder` field to try to release the lock: Test whether the lock is held by the current

thread, and if so, set `holder` to null; otherwise — if the lock is not held by this thread — throw an exception.

Since the `tryLock` call always returns immediately, also when it fails to take the lock (and returns false), entry to the critical section should be conditioned on the call returning true, like this:

```
if (lock.tryLock()) {
    ... critical section ...
    lock.unlock();
}
```

or this:

```
while (!lock.tryLock()) { }
... critical section ...
lock.unlock();
```

However, both of these are dubious approaches: The first one because it gives up taking the lock and executing the critical section if the lock is already taken; the second because it uses busy wait which may spend enormous amounts of compute power repeatedly calling `tryLock()` without getting any real work done. We will address this problem in implementations C and D, but first we show how to make the lock reentrant.

7.3 (B) ReentrantTryLock: non-blocking and reentrant

The `ReentrantTryLock` has an `AtomicReference` `holder` field as before and now also a volatile integer field `holdCount`. If the lock is available then `holder` is null, and `holdCount` does not matter. If some thread holds the lock, then `holder` refers to the `Thread` object corresponding to that thread, and `holdCount` is the number of times (> 0) that the thread has taken the lock:

```
class ReentrantTryLock {
    private final AtomicReference<Thread> holder
        = new AtomicReference<Thread>();
    private volatile int holdCount = 0;
    ...
}
```

Method `tryLock()` assigns the calling thread's `Thread` object to `current`. Then it tests whether the current thread already holds the lock, and if so increments `holdCount` and returns true. Otherwise it atomically tests whether the lock is available (when `holder` is null) and if so, sets `holder` to `current` and `holdCount` to 1, and returns true; otherwise it changes nothing and returns false, having failed to take the lock:

```
public boolean tryLock() {
    final Thread current = Thread.currentThread();
    if (holder.get() == current) {
        holdCount++;
        return true;
    } else if (holder.compareAndSet(null, current)) {
        holdCount = 1;
        return true;
    }
```

```

    }
    return false;
}

```

Method `unlock()` assigns the calling thread's `Thread` object to `current`. Then it tests whether the current thread already holds the lock, and if so decrements `holdCount`; if it is now zero, it sets `holder` to null and returns. If the current thread did not hold the lock, then the outer if condition was false and the method throws an exception:

```

public void unlock() {
    final Thread current = Thread.currentThread();
    if (holder.get() == current) {
        holdCount--;
        if (holdCount == 0)
            holder.compareAndSet(current, null);
        return;
    }
    throw new RuntimeException("Not lock holder");
}

```

Note that since `holder` and `holdCount` are private and manipulated only by `tryLock` and `unlock`, and each thread has a different `Thread` object, only one thread can execute the code after the test (`holder.get() == current`) at a time. Specifically, in `tryLock` only one thread can increment `holdCount` at a time, and no other thread can observe the value of `holdCount` between a successful `compareAndSet` and the assignment of 1 to `holdCount`. In `unlock`, only one thread can decrement `holdCount` at a time, and no thread can change `holder` between the `holder` test and the call to `compareAndSet`.

7.4 (C) SimpleLock: blocking and non-reentrant

Now we show how to make the non-blocking non-reentrant `SimpleTryLock` into a blocking non-reentrant `SimpleLock`. When the lock is not available, the `lock` method must now park the calling thread until the lock becomes available. Luckily, the Java class library provides a static method `LockSupport.park(lockObj)` from package `java.util.concurrent` that makes the current thread passive until a later call to `LockSupport.unpark(thr)` re-activates the thread `thr` and allows it to make a new attempt to take the lock.

In terms of Figure 7.1, `park(o)` moves the calling thread from the Running state to a “Parked o” state (not shown), similar to the “Locking o” state. Conversely, `unpark(thr)` moves thread `thr` from that state to the Enabled state, from where it may become Running (and attempt to obtain the lock in a new iteration of the `lock` while-loop shown below).

Class `SimpleLock` has a `holder` field as before, and also a field `waiters` which is a queue of the threads that have failed to take the lock and therefore are now parked (blocked), still hoping to take the lock:

```

class SimpleLock {
    private final AtomicReference<Thread> holder
        = new AtomicReference<Thread>();
    private final Queue<Thread> waiters
        = new ConcurrentLinkedQueue<Thread>();
}

```

```
    ...
}
```

Ignoring for now the complications related to handling of interrupts, we see that method `lock` below assigns the calling thread's `Thread` object to `current`, adds itself to the queue of threads hoping to take the lock, then enters a while-loop. If the current thread is not the first one in the `waiters` queue, or if it is the first one but the `compareAndSet` fails so the thread did not get the lock, then the thread parks itself, doing nothing at all until another thread calls `unpark(thr)` on this thread's `Thread` object, after which it evaluates the while-loop condition again. The '`this`' argument in `park(this)` is the `SimpleLock` object and tells which lock the parked thread hopes to take; this is useful for diagnosing deadlocks.

This while-loop may go on forever, but if it terminates, then we know the loop's condition became false. In this case it means that the current thread is the first one in the queue *and* it succeeded in taking the lock by updating `holder`. Therefore the current thread removes itself from the `waiters` queue and returns. Note that it does not return true as an indication of success; the very fact that `lock()` returns at all shows that it managed to take the lock.

Now what is the purpose of the `wasInterrupted` variable? While a thread `thr` is parked, another thread may call `thr.interrupt()` to interrupt it. This will not reactivate the parked thread, but it will set its interrupt status. The parked thread should not throw (and possibly handle) an `InterruptedException` until it has been reactivated, has succeeded in taking the lock, and has removed itself from the `waiters` queue. To arrange this, a thread that has been reactivated will call the `Thread.interrupted()` method to detect whether it was interrupted while parked and if so set `wasInterrupted` to true. If at some point the thread `thr` manages to take the lock, it will exit the while-loop, remove itself from the queue, and if `wasInterrupted` is true, interrupt itself.

```
public void lock() {
    final Thread current = Thread.currentThread();
    boolean wasInterrupted = false;
    waiters.add(current);
    while (waiters.peek() != current
           || !holder.compareAndSet(null, current)) {
        LockSupport.park(this);
        if (Thread.interrupted())
            wasInterrupted = true;
    }
    waiters.remove();
    if (wasInterrupted)
        current.interrupt();
}
```

Method `unlock` below assigns the calling thread's `Thread` object to `current`, then uses `compareAndSet` on the `holder` field to try to release the lock, setting `holder` to null. If this succeeds, it calls `unpark` on the first thread in the `waiters` queue, thereby possibly reactivating a thread that previously added itself to the queue and then parked itself. If the queue is empty, then `waiters.peek()` returns null but fortunately `unpark(null)` has no effect. If the `compareAndSet` fails, it is because the calling thread does not hold the lock, and in that case it throws an exception.

```
public void unlock() {
```

```

final Thread current = Thread.currentThread();
if (holder.compareAndSet(current, null))
    LockSupport.unpark(waiters.peek());
    throw new RuntimeException("Not lock holder");
}

```

The whole purpose of the **park-unpark** machinery is to avoid the waste of computing resources implied by busy wait in a while-loop as indicated in section 7.2 above. Instead of a while-loop checking the availability of the lock by calling **compareAndSet** a hundred million times per second, we use **park** and **unpark** to make sure the while-loop only calls **compareAndSet** when there is some chance that the lock has become available.

Indeed the **SimpleLock** is closely based on an example in the Java **LockSupport** documentation, which also says “*park serves as an optimization of a ‘busy wait’ that does not waste as much time spinning, but must be paired with an unpark to be effective.*”

7.5 (D) MyReentrantLock: blocking and reentrant

The final lock implementation **MyReentrantLock** is the one we actually want: it is both blocking and reentrant. It adds to implementation C the same **holdCount** machinery that implementation B added to implementation A.

The **MyReentrantLock** class has a **holder** field from implementation (A) **SimpleTryLock**, a volatile **holdCount** field from (B) **ReentrantTryLock**, and a **waiters** field from (C) **SimpleLock**, all serving the same purposes as before:

```

class MyReentrantLock {
    private final AtomicReference<Thread> holder
        = new AtomicReference<Thread>();
    private final Queue<Thread> waiters
        = new ConcurrentLinkedQueue<Thread>();
    private volatile int holdCount = 0;
    ...
}

```

Method **lock()** first tests whether the current thread already holds the lock, and if so increments **holdCount** and returns, as in class **ReentrantTryLock**. Otherwise it behaves just as method **lock** in class **SimpleLock**, except that it also sets **holdCount** to 1:

```

public void lock() {
    final Thread current = Thread.currentThread();
    if (holder.get() == current)
        holdCount++;
    else {
        boolean wasInterrupted = false;
        waiters.add(current);
        while (waiters.peek() != current
            || !holder.compareAndSet(null, current)) {
            LockSupport.park(this);
            if (Thread.interrupted())
                wasInterrupted = true;
        }
        holdCount = 1;
    }
}

```

```

        waiters.remove();
        if (wasInterrupted)
            current.interrupt();
    }
}

```

Method `unlock()` first tests whether the current thread already holds the lock, and if so decrements `holdCount`; if it is now zero, it sets `holder` to null, calls `unpark` on the first waiting thread, and returns. If the current thread did not hold the lock, then it throws an exception. This is as in `ReentrantTryLock` above, except for the `unpark` part which is from `SimpleLock`'s `unlock`.

```

public void unlock() {
    final Thread current = Thread.currentThread();
    if (holder.get() == current) {
        holdCount--;
        if (holdCount == 0) {
            holder.compareAndSet(current, null);
            LockSupport.unpark(waiters.peek());
        }
    } else
        throw new RuntimeException("Not lock holder");
}

```

7.6 Using a MyReentrantLock

As an example of the use of `MyReentrantLock`, consider a thread-safe long counter, whose `increment` and `get` methods can be called concurrently by any number of threads without lost updates or other problems:

```

static class LongCounterWithLock implements ILongCounter {
    private long count = 0;
    private final MyReentrantLock lock = new MyReentrantLock();
    public void increment() {
        lock.lock();
        try { count = count + 1; }
        finally { lock.unlock(); }
    }
    public long get() {
        lock.lock();
        try { return count; }
        finally { lock.unlock(); }
    }
}

```

The `get` method takes the lock for visibility of increments. The `compareAndSet` operations that implement the lock taking ensure that whatever is written to the `count` field by one thread will be visible to other threads subsequently calling `get`.

We use the `try-finally` template from section 7.1. It may not seem very useful here because `count = count + 1` cannot throw an exception, so

```
lock.lock(); count = count + 1; lock.unlock();
```

would actually work just as well. The `try-finally` structure is recommended anyway because if we later add further operations to the `increment` method body that may throw exceptions, then we might forget to add the `try-finally` structure, then lose unlocking, and run into an unexpected deadlock.

Moreover, in the `get` method the `try-finally` block has the subtle but necessary effect of calling `unlock` before the return is executed.